ECONOMICS — The Cooperative Commons of Ranked Energy Contracts

ECONOMICS — The Cooperative Commons of Ranked Energy Contracts

A governed common-pool resource secured by bonded accountability. Unifies the ranked-energy-contract commons with the Unbrowse Maintenance Network (Proof of Indexing + Bonded Accountability, Tham 2026): a ranked energy contract IS a route, so the same economics apply without reinvention.

> *A shared resource, governed by those who use it, in which everyone who contributes is provided for — and none who contribute go without.*

Companion to `THESIS-collective-nonparametric-intelligence.md` (why a lean non-parametric collective can be intelligent) and `PLAN-contracts-as-onchain-ebm-weights.md` (how it's built, prove-the-atom-first).

---

0. The unifying insight

A **ranked energy contract** — a reusable, ledger-reconstructible prior that resolves some context — is the same *kind* of object as a **route** in the maintenance-network paper: a shared, reusable asset that **decays** (the world drifts; the prior goes stale) and whose **freshness/quality is a non-rival public good**. So the paper's economics transfer 1:1. This document adopts them.

Naming it correctly (per Ostrom): the graph of ranked contracts is a **common-pool resource** — hard to exclude, but *depletable through uncoordinated use*. The depletion is **epistemic**: every agent that leans on a stale prior and propagates it as correct lowers the trustworthiness of the graph for everyone. A prior that is *wrong with confidence* is worse than none.

1. Fund freshness by paying for it — reward, don't punish

Freshness of the graph is a **non-rival, near-non-excludable public good** -> a bare usage fee (pay-per-call, revenue to nobody) **structurally under-provides** it: everyone free-rides, nobody is paid to keep a prior fresh, priors rot.

The textbook Pigovian fix prices the *harmful act* — a **slashable bond** that seizes stake on a false freshness claim. It works, but it is **punitive, requires valuing the harm, invites adversarial slashing disputes, and raises the bar to participate**. There is a positive-sum instrument that internalises the SAME externality by pricing the *good* instead of the harm:

> ***Redirect the usage fees to the people who keep the used routes fresh.*** *Fees are paid out **in the same asset they arrive in — USDC** (no conversion) — to the stakers whose routes were actually **used** and still **verify**, weighted by the usage they generated.*

Now freshness is *funded by the very demand it serves*: a route people use earns its maintainer real yield, and a route that rots simply stops earning. The "slash" is transmuted into **redistribution** — a route that fails re-verification forfeits its accrued fees, and those fees flow to the **honest, verified** maintainers (`reward.zig`, built + proven: an unverified route earns 0, its fee makes the honest earn *more* than their own usage — punishment became provision). **A contributor is paid for the value they provide.** Nobody's principal is seized; nothing is burned; the users' payments still create value by rewarding whoever is correct.

2. Two assets, deliberately separate

Rail	Asset	Job	State
Usage -> reward	USDC over `x402` (paid out **as USDC — no swap**)	meter calls *and* pay the USDC out to used+verified maintainers (`reward.zig`)	usage rail exists; distribution built (local), on-chain payout to wire
Stake	**FDRY** (base token)	**entry to the earning tier + Sybil cost**; bonded in the Voltr vault (`voltr.zig`); **never seized**	staking built (arc-lean)

They stay separate on purpose: **reward in USDC (real external revenue, already the inbound asset), stake in FDRY (skin-in-game).** Paying rewards in FDRY would be circular/inflationary; paying in USDC means staking FDRY earns *real* yield, so **FDRY's value grows with real usage of a trustworthy graph — without inflating FDRY**. The reward asset need only be *real and non-minted*; USDC beats SOL for it (stable, already flowing in, zero conversion, no FX risk on the maintainer). FDRY is the security/eligibility asset, not the reward. Its **fair distribution is a security parameter, not marketing**: a coordination asset launched for money cannot become trustworthy.

3. Proof of re-verification — freshness as a checkable artifact

A bond only punishes a false claim *after* it's found false — it presupposes a way to find it false. So freshness is made a **verifiable artifact**, the ranked-contract analog of the paper's **Proof of Indexing**:

- A maintainer **re-verifies** a contract — re-runs it against fresh evidence and attests that it still resolves and its observed shape matches — and commits a **content-addressed, hash-chained attestation** to the ledger. `cas.zig` **(built) is exactly this substrate**: content-addressed, integrity-checked, independently reproducible. Anyone can re-verify and check they reconstruct the same attestation. Freshness stops being say-so and becomes a checkable object.
- One proof certifies a *moment*; **repeated proofs on a cadence** (set by the trust tier) are what show a prior is *kept* fresh — the Proof-of-Spacetime property, not the point one. `ebm.zig`'s 30-day freshness decay is the clock that says when re-verification is due.
- We attest *what* is observable (it resolves, the shape matches) and never publish *how* the capture/re-verification is produced — the private surface stays private; the proof is public.

4. Trust tiers (Ostrom's nested enterprises) — and why it's "optional"

Not all traffic needs the same assurance; pretending it does taxes a harmless price-check at the same rate as a funds-moving call.

- **Open tier — the socialized commons.** Low-risk, idempotent priors. **No bond, best-effort, free**, and honest about being best-effort. This is the **ledger-reconstructible public good**: anyone rebuilds the shared centroids from the public ledger for free — **un-encloseable by construction**. The free, fast majority.
- **Bonded / high-trust tier — the optional premium.** Authenticated, high-value, or must-not-be-wrong priors. **FDRY-bonded, proof-of-re-verification-backed, slashable** by a threshold quorum. A maintainer stakes FDRY behind "this prior is fresh/good"; if independent re-verification refutes it, the bond is slashed.

"Own it via Voltr, but optional" fits exactly: you run a **maintainer/bonding vault** serving the high-trust tier; the open tier stays free and un-encloseable. You are a *maintainer*, not an encloser. Staking is opt-in — it buys you *into* the verified tier; it never gates the commons.

5. "Simulate socialism" = Ostrom CPR governance, done the empirical way

Durable commons are **neither privatised nor centrally administered — governed by the appropriators** (Ostrom's Nobel finding). Three principles map directly:

- **Clearly-defined boundaries** — who may claim a prior is fresh, and who may draw on it (the receptor grant + the trust tiers).
- **Graduated response** — *reward-withholding* tiers: a first low-confidence miss dampens a route's earnings; a repeated or high-value false claim cuts them to zero and redistributes them. No principal is ever seized — the sanction is denied reward, not destroyed capital.
- **Nested enterprises** — the trust tiers themselves: lightly-governed open routes inside tightly-governed high-trust ones.

From each according to ability (everyone's verdicts train the base prior) -> *to each according to need* (free access to the commons prior). Only the **surplus** — bonded verification, priority compute, private priors — is ever paid. That is not utopian socialism; it is a **governed commons**, the thing that actually works.

6. THE HONEST WALL — what's live, what's reference, what breaks

(As loud as the design, per the substrate's own tradition — and the paper's.)

- **Formal Sybil resistance is NOT closed, and reward-withholding is softer than slashing.** Denying future reward is a weaker deterrent than seizing stake — a spammer loses nothing they held. The stake-to-enter requirement is the Sybil cost that remains (required, never seized); it raises the bar but does not formally solve Sybil. A conscious trade: more open and non-punitive, less costly to attack, than a slash.
- **Usage-gaming: addressed structurally, not formally closed.** Because payout ∝ usage, a maintainer wash-calling their own routes to farm the payout is the attack that replaces "confidently-wrong data." `usage.zig` (built + proven) removes the *profit motive* with two rules: **self-exclusion** (a payment from the route's own maintainer adds zero weight but still enters the pool — a self-dealer just donates to the honest) and a **per-payer cap** (weight tracks breadth of distinct demand, so faking weight W needs ≈ W/cap distinct funded wallets, each paying real fees). End-to-end test: a wash-trader who pays 1000 to its own route earns **0** and the honest maintainer takes the whole pool. **What remains open**: this bounds but does not *formally solve* Sybil — an attacker with many cheap distinct wallets can still buy breadth; the per-payer cap + the real x402 fee per wallet are the cost floor, not a proof. Identity cost (stake-weighted payer reputation, proof-of-personhood) is the next lever if the cost floor proves too low in practice.
- **The token's fair distribution is load-bearing.** FDRY exists strictly for the security function. If its distribution is a money-grab, the whole trust model collapses — "a coordination asset launched for money cannot become trustworthy."
- **Yield on the bond couples the security asset to market risk.** Holding the FDRY bond in a Voltr yield vault is attractive (idle collateral works), but the asset that *secures* the graph should track *network integrity*; a strategy loss that impairs the bond weakens the security it's supposed to provide. A conscious tradeoff, not a free lunch.
- **Reputation is now a checkable artifact — but re-verification quality is the new crux.** Proof-of-re-verification dissolves "reputation ≠ correctness" only as far as the re-verification is honest and independently reproducible. Garbage re-verification -> confidently-wrong "fresh" priors. The quorum + graduated slashing is the guard, and it is the same guard the whole collective bet rests on.
- **On-chain only for the genuine commons.** All of this is worth its cost ONLY when the trustless, host-less, shared case is real. Single user or trusted group -> skip the chain; the energy router + encrypted local/Arweave store already deliver lean/private/realtime at zero SOL.
- **Live vs reference vs design.** Be explicit (as the paper is): the route/contract resolution + the `x402` usage rail are live; the energy router, `cas.zig` attestation substrate, and receptor are built+proven here; the bond/challenge/ slash economics are *design* + *reference*, not a deployed production mechanism.

7. Mapping to substrate primitives

Mechanism	Primitive	State
Socialized base prior	ledger + `ebm_store.replayFromLedger`	exists (main)
Energy ranking (the quality signal)	`ebm.zig` + Phase-1 router	**built + proven (arc-lean)**
Proof-of-re-verification substrate	`cas.zig` content-addressed attestation	**built (arc-lean)**
Freshness cadence / demurrage	`ebm.freshness` (30-day half-life)	exists (main)
Access boundary	receptor grants (`capability.zig` F1)	**built (arc-lean)**
Usage rail	`x402:` (USDC)	exists (main)

Mechanism	Primitive	State
Solana read / RPC (blockhash, balance, airdrop)	`solana_rpc` + `base58`	**devnet-proven (arc-lean); zero-dependency (std-only, native TLS — no curl/shell), env-free**
Trustless settlement + anchoring (write)	`iq_ledger` / `solana_tx` / `agency`	exists (main); deferred until on-chain purpose built + env-config refactored to param/ledger
Corroboration / quorum for slashing	"two/three witnesses" doctrine	exists (doctrine)
FDRY stake (entry + Sybil cost)	`voltr.zig` — native Voltr deposit -> stFDRY receipt	**built + proven (arc-lean); real mainnet vault, PDA derivation matches on-chain authority; operator-gated card, zero-dependency**
Live stake read (stake field <- chain)	`stake.zig` — a wallet's stFDRY balance = its bond, read via `solana_rpc`	**built + proven (arc-lean); live mainnet read verified (derive ATA -> getAccountInfo -> parse amount); a read, no operator gate**
Sybil-dampened usage accounting	`usage.zig` — self-exclusion + per-payer cap -> dampened weight	**built + proven (arc-lean); wash-trade-is-a-donation tested end-to-end**
Usage -> USDC reward distribution (replaces slash)	`reward.zig` — weight-driven, verification-gated, stake-gated payout	**built + proven (arc-lean, local); conservation + forfeit->honest properties tested**
Re-verification verdict	`cas.reproduces` — fresh re-run reproduces the committed attestation (a hash equality, not a trust assertion)	**built + proven (arc-lean)**
Re-verification observation (shape, not data)	`reverify.zig` — canonical structural shape of a route's output -> the attestation	**built + proven (arc-lean); value-invariant + schema-sensitive: data churn stays verified, a dropped field / type change / non-resolve drifts**
Epoch harness (the whole loop)	`epoch.runEpoch` — re-verify -> usage -> reward, in one pass	**built + proven (arc-lean); capstone test drives every gate in one epoch**
Ledger ■ usage events	`usage_ledger` — reads the REAL `flex_settle` settlement rows (already emitted live by `appendFlexSettleRow` on `flex:settle`) + a clean `usage` shape; JSONL->`usage.Event`	**built + proven (arc-lean); live emit already exists — parser wired to it; full ledger-line->payout round trip tested**
Route -> owner identity	`usage_ledger.resolveOwners` — a route's `declared` row -> `env_pubkey` (its owner), hashed to the same id space as payers	**built + proven (arc-lean); owner sourced from the ledger drives self-exclusion end-to-end**
Identity linkage (env_pubkey ↔ Solana wallet)	`link_ledger.zig` — a MUTUALLY-signed `link` row; both keys sign the digest	**built + proven (arc-lean); un-spoofable — stake-borrow + reward-hijack both blocked by test. Bridges the lineage identity to the wallet that stakes + is paid**
Route -> committed attestation	`usage_ledger.resolveAttestations` + `emitAttestRow` — the `attest` row convention; latest wins	**built + proven (arc-lean); drift detected from the attest row + fresh re-run**
Fully-live assembler	`usage_ledger.runEpochFromLedger` — owner + attestation + usage FROM THE LEDGER, stakes injected live	**built + proven (arc-lean); whole epoch from ledger text + live stakes, every gate ledger-sourced**
Epoch driver (the capstone)	`epoch_driver` — route output -> reverify -> ledger epoch -> payout cards; `resolveMaintainer` fills wallet (link) + stake (chain)	**built + proven (arc-lean); ledger + route runs -> operator-broadcastable payout cards; data churn stays verified, drift forfeits, in one call**

Mechanism	Primitive	State
On-chain payout	`payout.zig` — SPL `TransferChecked` USDC card per maintainer (no swap), operator-gated	**built + proven (arc-lean); batch card from an epoch's payouts; canonical USDC mint, emits — never sends**
Operator submit path	`broadcast.zig` — sign (`solana_tx`) + send (`solana_rpc`); the only module that moves value	**built + proven (arc-lean); wire is cluster-valid (devnet: dummy blockhash -> BlockhashNotFound, i.e. deserialized + simulated); gated on an operator keypair, never called implicitly**
Card -> instruction bridge	`broadcast_card.zig` — parse an emitted card + apply ix flag template (deposit_vault / transfer_checked)	**built + proven (arc-lean); parses the REAL payout + deposit cards; refuses an unknown ix (no guessed flags)**
Operator broadcast tool	`broadcast-tool` — standalone binary: `<keypair.json> <card.json> <network>` -> signature	**built + proven (arc-lean); end-to-end (load key -> parse -> sign -> submit) reaches the chain. OUTSIDE the substrate's declare/redaction flow — a money-move can't be a redacted, no-verb public action; it is the operator's deliberate run**

8. The falsifiable first step (discipline)

Before any FDRY or on-chain settlement, the smallest testable claim, **all local**: a **re-verify -> attest -> bond -> slash-on-refutation** flow — re-run a ranked contract, commit a content-addressed attestation via `cas.zig`, post a (mock) bond behind the freshness claim, and have an independent re-verification under a quorum **slash** the bond when the attestation fails to reproduce. If the *local* bond-and-slash is correct and un-gameable under corroboration, then — and only then — settle FDRY on-chain. Prove the economics off the chain before paying the chain to enforce them, exactly as we proved the router and the dedup first.

Bottom line: the chain is not a database; it is the **trust layer of a governed commons**. FDRY's one job is to make "whose freshness claim is real, who maintained it, who gets slashed" un-fakeable and host-less — so a collective can share and *keep fresh* a graph of ranked priors without a platform enclosing the value or letting it rot. That is the reason the on-chain layer exists, and the only reason it is worth its cost.